TP3 – OpenMP (Introduction)

Khaoula Mejhoudi

February 2026

1 Exercise 1: Hello World

The goal is to print each thread's rank and the total number of threads.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int nb;
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        nb = omp_get_num_threads();
        printf("Hello from the rank %d thread \n", id);
    }
    printf("Parallel execution of hello world with %d threads \n", nb);
    return 0;
}
```

Inside the `#pragma omp parallel` block, each thread gets its own `id` (declared locally, so private). The variable `nb` is shared and gets the total thread count.

I tested with different values of `OMP_NUM_THREADS` (2, 4, 8). The thread order in the output changes every run since there is no guaranteed execution order.

2 Exercise 2: Parallelizing PI Calculation

We start from the serial numerical integration of π:

$$\pi = \int_0^1 \frac{4}{1+x^2} dx \approx \sum_{i=0}^{N-1} \frac{4}{1+x_i^2} \cdot \Delta x$$

The constraint is to use `#pragma omp parallel` (not `parallel for`). I used `#pragma omp for` with a `reduction` clause inside the parallel region:

```
#pragma omp parallel
{
```

```
    #pragma omp for reduction (+:sum)
        for (i = 0; i < num_steps; i++) {
            x = (i + 0.5) * step;
            sum = sum + 4.0 / (1.0 + x * x);
        }
}
pi = step * sum;
```

The `reduction(+:sum)` clause gives each thread a private copy of `sum`, and at the end OpenMP adds them all together. This avoids race conditions without manual locking.

Timing is done with `omp_get_wtime()` around the parallel region. The result is $\pi = 3.141593$, which is correct.

3 Exercise 3: PI with Loops

The goal is to parallelize the serial PI program by adding only one line. We simply add `#pragma omp parallel for reduction(+:sum)` before the for loop:

```
#pragma omp parallel for reduction(+:sum)
for (i = 0; i < num_steps; i++) {
    x = (i + 0.5) * step;
    sum = sum + 4.0 / (1.0 + x * x);
}
```

This is the combined form of what we did in Exercise 2. One line is enough because `parallel for` handles thread creation and work distribution automatically.

4 Exercise 4: Matrix Multiplication

We parallelize $C = A \times B$ with $m = n = 1000$.

4.1 Part 1: `parallel for`

Adding `#pragma omp parallel for` before the outer `i` loop is enough. Each thread computes a subset of rows of C, and since different threads write to different elements of C, there is no race condition.

```
#pragma omp parallel for
for (int i = 0; i < m; i++) {
    for (int j = 0; j < m; j++) {
        for (int k = 0; k < n; k++) {
            c[i * m + j] += a[i * n + k] * b[k * m + j];
        }
    }
}
```

Note: These results were obtained on a 2-node session, which introduces inter-node overhead. The later results use a 1-node session.

Threads	Time (s)	Speedup	Efficiency
1	8.463	1.00	1.00
4	1.739	4.87	1.22
8	1.083	7.81	0.98
16	0.812	10.42	0.65
36	0.551	15.36	0.43

Table 1: Results with `parallel for` (2 nodes, 36 cores)

4.2 Part 2: `collapse(2)`

With `collapse(2)`, OpenMP merges the `i` and `j` loops into a single loop of $m \times m = 10^6$ iterations. We do not collapse the `k` loop because it accumulates into the same `c[i][j]` element, which would cause a race condition.

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < m; i++) {
    for (int j = 0; j < m; j++) {
        ...
```

Threads	Time (s)	Speedup	Efficiency
1	5.501	1.00	1.00
4	1.448	3.80	0.95
8	0.698	7.88	0.98
16	0.351	15.67	0.98
36	0.159	34.60	0.96

Table 2: Results with `collapse(2)` (1 node, 36 cores)

The collapse version shows near-ideal scaling. However, we should note that the two sets of results were obtained on different configurations (2 nodes vs 1 node), so the improvement is partly due to eliminating inter-node communication overhead.

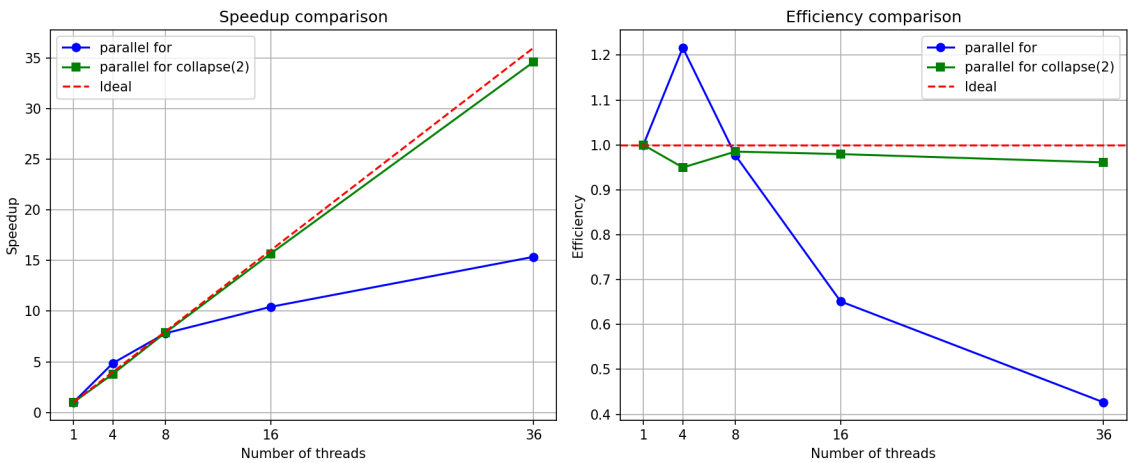

Figure 1: Speedup and efficiency: `parallel for` vs `collapse(2)`

4.3 Part 3: Schedule Modes

To test different scheduling strategies without recompiling, I used `schedule(runtime)` and set the `OMP_SCHEDULE` environment variable. Results with 36 threads:

Chunk size	static (s)	dynamic (s)	guided (s)
1	0.369	0.353	0.366
10	0.393	0.391	0.377
50	0.561	0.556	0.556
100	0.561	0.560	0.562

Table 3: Schedule modes with different chunk sizes (36 threads)

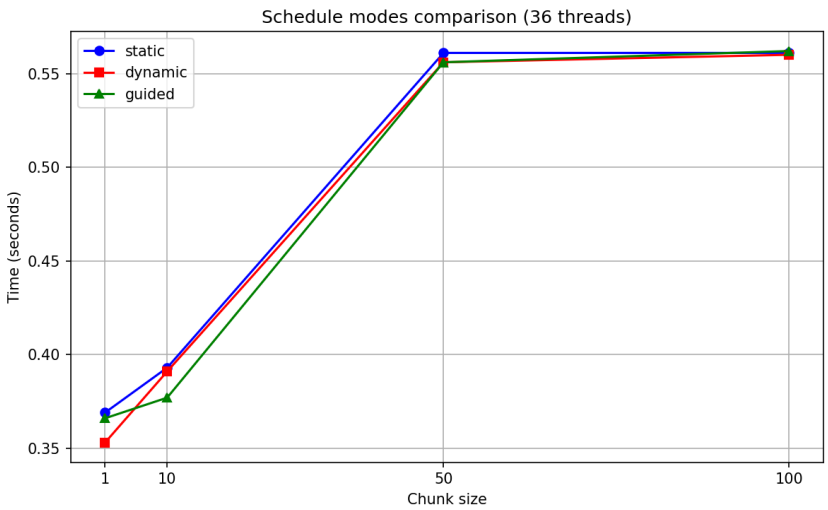

Figure 2: Schedule modes comparison

All three modes give similar performance because matrix multiplication has a uniform workload per iteration. Small chunks (1, 10) perform better than large ones (50, 100) because with 36 threads and 1000 iterations, large chunks cause uneven distribution and some threads get more work than others.

5 Exercise 5: Jacobi Method

The Jacobi method solves $Ax = b$ iteratively. Each iteration computes a new approximation independently for each element, making the inner loop parallelizable.

I added two pragmas inside the `while` loop:

```
// Loop 1: compute new approximation
#pragma omp parallel for private(j)
for (i = 0; i < n; i++) {
    x_courant[i] = 0;
    for (j = 0; j < i; j++)
        x_courant[i] += a[j * n + i] * x[j];
    for (j = i + 1; j < n; j++)
```

```
        x_courant[i] += a[j * n + i] * x[j];
    x_courant[i] = (b[i] - x_courant[i]) / a[i * n + i];
}

// Loop 2: compute convergence norm
double absmax = 0;
#pragma omp parallel for reduction(max:absmax)
for (i = 0; i < n; i++) {
    double curr = fabs(x[i] - x_courant[i]);
    if (curr > absmax) absmax = curr;
}
```

Problem size matters: With the default $n = 120$, the program got *slower* with more threads because the overhead of creating threads at every `while` iteration dominated the tiny computation. After increasing to $n = 500$ (compiled with `-DVAL_N=500 -DVAL_D=500`), the results became logical:

Threads	CPU time (s)	Speedup	Efficiency
1	5.488e-02	1.00	1.00
4	1.405e-02	3.91	0.98
16	8.258e-03	6.64	0.42
36	9.502e-03	5.78	0.16

Table 4: Jacobi method results ($n = 500$)

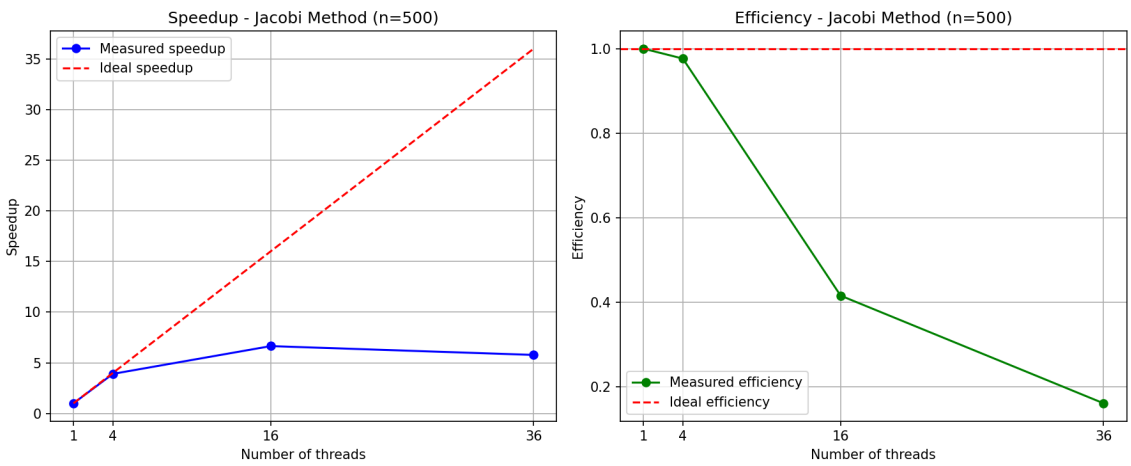

Figure 3: Speedup and efficiency for the Jacobi method ($n = 500$)

The speedup is good up to 16 threads but drops at 36. Even with $n = 500$, the problem is still relatively small for 36 threads. A larger n would show better scaling.